

Examen Final: Estructuras de Datos 2024

Tema Central: Universo Saga Cinematográfica de Matrix

Instrucciones Generales: Responde todas las preguntas en base al contexto presentado utilizando exclusivamente Python para las partes prácticas.

Parte Teórica

1. Encapsulamiento e Interfaces (Unidad 1)

- **Respuesta:** En Programación Orientada a Objetos (POO), una interfaz define un contrato que especifica qué métodos deben implementarse, pero no cómo. Para representar el programa que controla la entrada y salida en la Matrix, se podría usar una interfaz con métodos como `autorizar_entrada(usuario)` y `registrar_salida(usuario)`. Cada implementación concreta puede manejar estos métodos de forma diferente, manteniendo el encapsulamiento al ocultar los detalles internos.

2. Estructuras de Datos Recursivas (Unidad 2)

- **Respuesta:** Un grafo es una estructura de datos compuesta por nodos (o vértices) conectados mediante aristas. Dos representaciones comunes son:
 - **Lista de adyacencia:** Representa cada nodo con una lista de sus vecinos. Es eficiente en términos de espacio para grafos dispersos.
 - **Matriz de adyacencia:** Usa una matriz para almacenar las conexiones. Es más adecuada para grafos densos porque permite acceso rápido a las conexiones.

3. Análisis de Algoritmos (Unidad 6)

- **Respuesta:** La notación $O()$ describe la complejidad asintótica de un algoritmo en función de su entrada. Se usa para clasificar el tiempo de ejecución:
 - **Mejor caso:** Tiempo mínimo requerido.
 - **Caso promedio:** Tiempo esperado.
 - **Peor caso:** Tiempo máximo.
En una búsqueda lineal, por ejemplo, el mejor caso es $O(1)$ (primer elemento es el deseado), el caso promedio es $O(n/2)$, y el peor caso es $O(n)$ (último elemento).

4. Problemas NP (Unidad 10)

- **Respuesta:** El problema del camino más corto no es NP-completo, ya que se puede resolver en tiempo polinómico para grafos ponderados con algoritmos como Dijkstra. Sin embargo, encontrar caminos óptimos en problemas más complejos, como el viajante de comercio, sí pertenece a NP-completo. Dijkstra encuentra el camino más corto mediante una búsqueda codiciosa, calculando distancias acumulativas desde el nodo origen y actualizándolas iterativamente.

Parte Práctica

1. Árboles Binarios (Unidad 3)

```
class Nodo:
    def __init__(self, valor):
        self.valor = valor
        self.izq = None
        self.der = None

def inorden(nodo):
    if nodo:
        inorden(nodo.izq)
        print(nodo.valor, end=" ")
        inorden(nodo.der)

# Construcción del árbol
raiz = Nodo("Inicio")
raiz.izq = Nodo("Píldora Roja")
raiz.der = Nodo("Píldora Azul")

print("Recorrido InOrden del árbol:")
inorden(raiz)
```

2. Heap Binaria y Cola de Prioridades (Unidad 5)

```
import heapq

class HeapBinario:
    def __init__(self):
        self.heap = []

    def insertar(self, elemento):
        heapq.heappush(self.heap, -elemento) # Usar negativos para simular un max-heap

    def extraer_maximo(self):
        return -heapq.heappop(self.heap) # Convertir de nuevo a positivo

# Prueba del heap
heap = HeapBinario()
heap.insertar(10)
heap.insertar(20)
heap.insertar(15)

print("Elemento máximo extraído:", heap.extraer_maximo())
```

3. Algoritmos de Recorrido (Unidad 8)

```

from collections import deque

def bfs(grafo, inicio):
    visitados = set()
    cola = deque([inicio])
    while cola:
        nodo = cola.popleft()
        if nodo not in visitados:
            print(nodo, end=" ")
            visitados.add(nodo)
            cola.extend(grafo[nodo])

# Grafo de ejemplo
grafo = {
    'A': ['B', 'C'],
    'B': ['D'],
    'C': ['E'],
    'D': ['E'],
    'E': []
}

print("Recorrido BFS:")
bfs(grafo, 'A')

```

4. Ordenamiento Topológico (Unidad 9)

```

def orden_topologico(grafo):
    def dfs(nodo):
        if nodo not in visitados:
            visitados.add(nodo)
            for vecino in grafo[nodo]:
                dfs(vecino)
            resultado.append(nodo)

    visitados = set()
    resultado = []
    for nodo in grafo:
        dfs(nodo)
    return resultado[::-1]

# Grafo dirigido de ejemplo
grafo = {
    'A': ['B', 'C'],
    'B': ['D'],
    'C': ['D'],
    'D': ['E'],
    'E': []
}

print("Ordenamiento topológico:", orden_topologico(grafo))

```

5. Heap Binario para Planificación

```
import heapq

class Monticulo:
    def __init__(self):
        self.heap = []

    def insertar(self, tarea, prioridad):
        heapq.heappush(self.heap, (prioridad, tarea))

    def extraer_minimo(self):
        return heapq.heappop(self.heap)

# Prueba del montículo
monticulo = Monticulo()
monticulo.insertar("Tarea A", 5)
monticulo.insertar("Tarea B", 3)
monticulo.insertar("Tarea C", 8)
monticulo.insertar("Tarea D", 1)

print("Tareas en orden de prioridad:")
while monticulo.heap:
    print(monticulo.extraer_minimo())
```

6. Ruta Mínima en Grafos

```
import heapq

def dijkstra(grafo, inicio):
    distancias = {nodo: float('inf') for nodo in grafo}
    distancias[inicio] = 0
    heap = [(0, inicio)]

    while heap:
        distancia_actual, nodo_actual = heapq.heappop(heap)

        if distancia_actual > distancias[nodo_actual]:
            continue

        for vecino, peso in grafo[nodo_actual]:
            distancia = distancia_actual + peso
            if distancia < distancias[vecino]:
                distancias[vecino] = distancia
                heapq.heappush(heap, (distancia, vecino))

    return distancias
```

```
# Grafo ponderado de ejemplo
grafo = {
    1: [(2, 2), (3, 4)],
    2: [(3, 1), (4, 7)],
    3: [(5, 3)],
    4: [(5, 1)],
    5: []
}

print("Distancias desde el nodo 1:", dijkstra(grafo, 1))
```

Criterios de Evaluación:

- Completitud y claridad de las respuestas.
- Correcta aplicación de los conceptos teóricos.
- Precisión y eficiencia en las implementaciones prácticas.
- Uso correcto del pseudocódigo o Python.

Duración del Examen: 3 horas.

Material Permitido: Apuntes y libros indicados en la bibliografía obligatoria del programa.

¡Buena suerte! "La elección, como siempre, está en tus manos."